Sprint 15: Testing Strategy (ටෙස්ටිං උපාය මාර්ග)

මෙම අධ්‍යයන සටහනෙන් Sprint 15 හි අඩංගු සියලුම සිද්ධාන්ත, සංකල්ප සහ ප්‍රායෝගික උදාහරණ ඉතාමත් සරල මට්ටමින් සාකච්ඡා කෙරේ.

0. Where You Are Before Sprint 15 (Sprint 15 ට පෙර ඔබ දැනගත යුතු දෑ)

- **Concept Explanation (Sinhala):** Sprint 15 ආරම්භ කිරීමට පෙර ඔයාට React පිළිබඳව සහ මූලික ටෙස්ටිං පිළිබඳව මෙන්න මේ දේවල් ගැන පැහැදිලි අවබෝධයක් තියෙන්න ඕනේ:
  - React ඇප්ස් හදන්නේ කුඩා **Components** එකතු කිරීමෙන් බව.
  - **JSX** මඟින් ස්ක්‍රීන් එකේ පෙන්විය යුතු දේ විස්තර කරන බව.
  - Parent component එකක සිට child component එකකට දත්ත යවන්නේ **Props** හරහා බව.
  - වෙනස් වන දත්ත මතක තබා ගැනීමට **State** පාවිච්චි කරන බව.
  - Click කිරීම්, Type කිරීම් සහ Form submit කිරීම් වලට ප්‍රතිචාර දක්වන්න **Events** පාවිච්චි කරන බව.
  - **Effects (useEffect)**, දත්ත ලබා ගැනීම (**fetching**), **routing**, **Context**, සහ මූලික performance දේවල් ගැන ඔයා දැනටමත් දන්නවා.
  - Sprint 14 මඟින් ඔයාට ටෙස්ටිං වල පදනම හඳුන්වා දුන්නා.
  - Manual testing (මිනිසෙකු විසින් කරන) සහ Automated testing (පරිගණක වැඩසටහන් මඟින් කරන) අතර වෙනස ඔයා දන්නවා.
  - **Unit test** එකක් මඟින් කේතයේ ඇති ඉතා කුඩා කොටසක් (සිංගල් ෆන්ක්ෂන් එකක් වැනි) පමණක් පරීක්ෂා කරන බව ඔයා දන්නවා.
  - `test()`, `expect()` සහ විවිධ matchers වල මූලික syntax ඔයා දන්නවා.
  - **React Testing Library** මඟින් යූසර් ඇත්තටම දකින සහ කරන දේවල් ටෙස්ට් කිරීමට දිරිගන්වන බව ඔයා දන්නවා.

  Sprint 14 සහ Sprint 15 අතර ප්‍රධාන වෙනස:

  - **Sprint 14 ඇසූ ප්‍රශ්නය:** "මට මූලික ටෙස්ට් එකක් ලියාගන්න පුළුවන්ද?"
  - **Sprint 15 අසන වඩාත් උපායමාර්ගික ප්‍රශ්නය:** "මෙම අවදානම (risk) මඟහරවා ගැනීමට මම කුමන ටෙස්ට් වර්ගයද (test type) භාවිතා කළ යුත්තේ?"

  සරලවම කිවහොත්, Sprint 15 යනු තවත් `expect()` කෝඩ් ලියන එක ගැනම නෙවෙයි. එයින් උගන්වන්නේ නිවැරදි ප්‍රශ්නයට නිවැරදි ටෙස්ට් ක්‍රමය තෝරාගන්නේ කෙසේද යන්නයි.

1. Sprint 15 Goal (අපගේ ඉලක්කය)

- **Concept Explanation (Sinhala):** Sprint 15 හි ප්‍රධානතම ඉලක්කය වන්නේ ඇප් එකක සිදුවිය හැකි විවිධ අවදානම් (risks) හඳුනාගෙන, ඒ ඒ අවදානමට ගැළපෙනම ටෙස්ට් වර්ගය (test type) තෝරාගැනීමට ඉගෙන ගැනීමයි. අපි අහඹු ලෙස (randomly) ටෙස්ට් කරන්නේ නැතිව, ක්‍රමවත්ව අවදානම මත පදනම්ව ටෙස්ට් කිරීමට මෙහිදී හුරු වෙනවා.

5. Big Mental Model & Test Choice Map (ප්‍රධාන මානසික ආකෘතිය සහ ටෙස්ට් තේරීම් සිතියම)

- **Concept Explanation (Sinhala):** ආධුනිකයෙක් කරන ලොකුම වැරැද්දක් තමයි හැම ප්‍රශ්නයකටම එකම වර්ගයේ ටෙස්ට් එකක් ලියන්න උත්සාහ කිරීම. නමුත් නිවැරදි ක්‍රමය වන්නේ මුලින්ම අවදානම (Risk) හඳුනාගෙන, ඊට ගැළපෙනම ටෙස්ට් වර්ගය තෝරාගෙන ක්‍රියාත්මක කිරීමයි.

  පහත සිතියම (Map) මඟින් විවිධ ගැටලු සඳහා ගැළපෙනම ටෙස්ට් වර්ගය පෙන්වා දෙයි:

  - Logic problem (තර්කන ගැටලුවක්) ➡️ Unit test
  - Feature problem (ෆීචර් එකක හැසිරීමේ ගැටලුවක්) ➡️ Functional test
  - Basic health problem (ඇප් එකේ මූලික පණ ගැන්වීමේ ගැටලුවක්) ➡️ Smoke test
  - Journey problem (සම්පූර්ණ පරිශීලක ගමනක ගැටලුවක්) ➡️ E2E test
  - User confusion (ඩිසයින් එක නිසා පරිශීලකයා අතරමං වීම) ➡️ Usability test
  - Device/browser problem (විවිධ බ්‍රවුසර් හෝ ෆෝන් වල ගැටලු) ➡️ Compatibility test
  - Speed/load problem (වේගය සහ වැඩි ට්‍රැෆික් දරාගැනීමේ ගැටලු) ➡️ Performance test
  - Protection problem (දත්ත ආරක්ෂාව සහ හැකින් ප්‍රහාර) ➡️ Security test
  - Fixed bug verification (හැදූ බග් එකක් ආයෙත් හරියට වැඩද බැලීම) ➡️ Retesting
  - Old feature breaking (අලුත් කෝඩ් දැම්මම පරණ ඒව කැඩීම) ➡️ Regression test
  - Version comparison (ඩිසයින් දෙකක් සංසන්දනය කිරීම) ➡️ A/B test
  - Pre-release selected testers (එළියට දාන්න කලින් තෝරාගත් අය ලවා ටෙස්ට් කරවීම) ➡️ Alpha test
  - Pre-release real users (ලෝන්ච් කරන්න කලින් සැබෑ යූසර්ස්ලා ලවා ටෙස්ට් කරවීම) ➡️ Beta test

Part One: Functional and Non-Functional Testing

7. Functional Testing (ෆන්ක්ෂනල් ටෙස්ටිං)

- **Concept Explanation (Sinhala): Functional testing** මඟින් පරීක්ෂා කරන්නේ ඇප් එකේ යම් විශේෂාංගයක් (feature) බලාපොරොත්තු වූ පරිදි ක්‍රියා කරනවාද යන්නයි. සරලවම කිවහොත්, **"ඇප් එක කුමක් කරන්නේද?" (What the app does)** යන්න මෙයින් බලයි.

  උදාහරණ:

  - යූසර් කෙනෙකුට සාර්ථකව log in විය හැකිද?
  - Search box එකේ ටයිප් කළ විට කෝස් ලිස්ට් එක filter වෙනවාද?
  - "Enroll" බටන් එක ක්ලික් කළ විට කෝස් එක එකතු වෙනවාද?

  React වලදී, අපි බොහෝ විට කරන්නේ යූසර්ගේ පැත්තෙන් පෙනෙන හැසිරීම් (visible behavior) ටෙස්ට් කිරීමයි.

- **Code Block:**

```
// React Testing Library භාවිතයෙන් ලියන ලද Functional Test එකක හැඩය
render(<Search courses={courses} />);

await user.type(screen.getByLabelText("Search"), "React");

expect(screen.getByText("React Basics")).toBeInTheDocument();
expect(screen.queryByText("Vue Basics")).not.toBeInTheDocument();
```

- **Concept Explanation (Sinhala):** ඉහත ටෙස්ට් එකෙන් අසන්නේ: "යූසර් React කියා සර්ච් කළ විට, ඔහුට React පාඩම් පෙන්වා, Vue පාඩම් සඟවනවාද?" යන්නයි. මෙය පැහැදිලිවම **Functional test** එකකි.

  Unit test එකක්ද Functional test එකක කොටසකි: කුඩා pure function එකක් පරීක්ෂා කරන Unit test එකක්ද මූලිකවම කරන්නේ එහි ක්‍රියාකාරීත්වය (functionality) නිවැරදිදැයි බැලීමයි (උදා: `expect(formatPrice(4)).toBe("$4.00")`).

8. Non-Functional Testing (නන්-ෆන්ක්ෂනල් ටෙස්ටිං)

- **Concept Explanation (Sinhala): Non-functional testing** මඟින් පරීක්ෂා කරන්නේ ඇප් එකේ ක්‍රියාකාරීත්වයේ ගුණාත්මකභාවය (quality) යි. එනම්, **"ඇප් එක කොතරම් හොඳින් වැඩ කරන්නේද?" (How well the app behaves)** යන්නයි.

  මෙහිදී අසන ප්‍රශ්න:

  - ඇප් එක වේගවත්ද? (Speed)
  - ඇප් එක පාවිච්චි කරන්න ලෙහෙසිද? (Usability)
  - ඇප් එක ආරක්ෂිතද? (Security)
  - විවිධ බ්‍රවුසර් වල වැඩ කරනවාද? (Compatibility)
  - ලොකු ට්‍රැෆික් එකක් දරාගත හැකිද? (Performance/Scalability)

  සරල සැසඳීමක්:

  - **Functional:** Login form එකෙන් නිවැරදි යූසර් කෙනෙකුට ඇතුල් විය හැක.
  - **Non-Functional:** එම Login form එක අධික ට්‍රැෆික් එකක් යටතේ වුවද ඉතා වේගයෙන් සහ ආරක්ෂිතව වැඩ කරයි.

Part Two: Smoke Testing and Login Scenarios

9. Smoke Testing (ස්මෝක් ටෙස්ටිං)

- **Concept Explanation (Sinhala): Smoke test** එකක් යනු ඇප් එකක් අලුතින් ඩිප්ලෝයි (deploy) කළ පසු හෝ බිල්ඩ් (build) එකක් හැදූ පසු එහි ප්‍රධාන කොටස් පැහැදිලිවම බිඳ වැටී නැතිදැයි බලන ඉතා වේගවත්, මූලික ආරක්ෂක පියවරකි (quick basic check).

  මෙම නම හැදුණේ ඉලෙක්ට්‍රොනික උපකරණයක් ප්ලග් එකට ගැසූ සැනින් දුමක් (smoke) එනවාද කියා බලන සරල ක්‍රමවේදයෙනි. ඇප් එකකදී, ඇප් එක ක්‍රෑෂ් නොවී (crash) ප්‍රධාන පිටු ටික ලෝඩ් වෙනවාද යන්න පමණක් මෙයින් බලයි.

  Smoke test එකකින් පරීක්ෂා කරන දේ:

  1. ඇප් එක ඕපන් වීම.
  2. Home page එක ලෝඩ් වීම.
  3. Login page එක ඕපන් වීම.
  4. ප්‍රධාන Heading එක දිස්වීම.
  5. කිසිදු crash screen එකක් නොපෙන්වීම.

  මතක තබා ගන්න: Smoke test එකක් පාස් වූ සැනින් මුළු ඇප් එකම 100% පරිපූර්ණ යැයි කිව නොහැක. එයින් අදහස් වන්නේ ඇප් එක ගැඹුරින් ටෙස්ට් කිරීමට තරම් ස්ටේබල් (stable) මට්ටමක ඇති බව පමණි.

10. Login Scenario Testing (ලොගින් අවස්ථා ටෙස්ට් කිරීම)

- **Concept Explanation (Sinhala):** එකම Login විශේෂාංගයකට (feature) වුවද, එහි ඇති විවිධ අවදානම් (risks) වලට අනුව වෙනස් වෙනස් ටෙස්ට් ක්‍රම භාවිතා කළ හැක:
  - **10.1 Functional Login Test:** නිවැරදි විස්තර ඇතුල් කළ විට ලොගින් එක වැඩ කරනවාද?
  - **10.2 Validation Login Test:** වැරදි විස්තර හෝ හිස් ඉන්පුට් දැමූ විට නිවැරදිව error messages පෙන්වනවාද?
  - **10.3 Security Login Test:** Logout වී සිටින යූසර් කෙනෙකුට කෙලින්ම URL එක ගසා `/dashboard` පිටුවට යාමට නොහැකි වන ලෙස වළක්වා තිබේද?
  - **10.4 E2E Login Flow:** යූසර් කෙනෙකුට බ්‍රවුසර් එක හරහා සැබෑ ලෙසම මුළු ලොගින් ක්‍රියාවලියම අවසන් කළ හැකිද?

Part Three: E2E Testing and BDD

11. E2E Testing (End-to-End ටෙස්ටිං)

- **Concept Explanation (Sinhala): E2E (End-to-End) testing** මඟින් කරන්නේ සැබෑ පරිශීලකයෙකු ඇප් එක ආරම්භයේ සිට අවසානය දක්වාම පාවිච්චි කරන සම්පූර්ණ ගමනක් (complete user flow/journey) බ්‍රවුසර් එකක් ඔටෝමේට් කිරීම මඟින් පරීක්ෂා කිරීමයි.

  යූසර් ජර්නි සඳහා උදාහරණ:

  - Register වීම ➡️ Login වීම ➡️ Dashboard එක බැලීම.
  - Course එකක් සර්ච් කිරීම ➡️ විස්තර බැලීම ➡️ Enroll වීම.
  - බඩු කාර්ට් එකට දැමීම ➡️ Checkout වීම ➡️ Confirmation පිටුව බැලීම.

  E2E ටෙස්ට් වැදගත් වන්නේ ඇයි? එය ඇප් එකේ ඇති සියලුම ස්තර (layers) එකට එකතු වී වැඩ කරන ආකාරය පරීක්ෂා කරයි (Routes, Forms, APIs, Database, UI, Auth).

  E2E ටෙස්ට් මිල අධික (expensive) වන්නේ ඇයි?

  - රන් වීමට වැඩි වෙලාවක් යයි (Slow).
  - Setup කිරීමට අපහසුය.
  - UI එක පොඩ්ඩක් වෙනස් වූ සැනින් ටෙස්ට් එක ෆේල් විය හැක (Fragile).
  - පරීක්ෂා කිරීමට සැබෑ දත්ත (realistic test data) අවශ්‍ය වේ.

  හොඳ E2E අපේක්ෂකයින් (Good Candidates): Signup, Login, Checkout, Enrollment, Payment වැනි කැඩුණහොත් ව්‍යාපාරයට හෝ යූසර්ට සෘජුවම හානි වන ප්‍රධාන ගමන් මඟවල්.

  නරක E2E අපේක්ෂකයින් (Poor Candidates): `formatPrice()`, `capitalizeName()` වැනි කුඩා pure logic හෝ helper functions. මේවා Unit tests මඟින් ටෙස්ට් කළ යුතුය.

- **Code Block:**

```
// Playwright භාවිතයෙන් ලියන ලද සරල E2E ටෙස්ට් එකක හැඩය
import { test, expect } from "@playwright/test";

test("user can open products page", async ({ page }) => {
  await page.goto("/products");
  await expect(
    page.getByRole("heading", { name: "Products" })
  ).toBeVisible();
});
```

12. BDD Scenario Shape (BDD ආකාරයේ අවස්ථා)

- **Concept Explanation (Sinhala): BDD (Behavior-Driven Development)** මඟින් ඇප් එකක හැසිරීම තාක්ෂණික නොවන ඕනෑම කෙනෙකුට තේරෙන සරල මිනිස් භාෂාවෙන් ලියා දක්වයි. මේ සඳහා ප්‍රධාන වශයෙන් **Given/When/Then** ව්‍යුහය භාවිතා කරයි:
  - **Given:** යම් ආරම්භක තත්ත්වයක් (starting situation).
  - **When:** පරිශීලකයා කරන යම් ක්‍රියාවක් (user action).
  - **Then:** බලාපොරොත්තු වන අවසාන ප්‍රතිඵලය (expected result).

  BDD උදාහරණයක්:

  - **Given** යූසර් Course List පේජ් එකේ ඉන්න කොට
  - **When** යූසර් "React" කියලා සර්ච් කරන කොට
  - **Then** ස්ක්‍රීන් එකේ පෙන්විය යුත්තේ "React" වලට ගැළපෙන කෝස් විතරයි.

Part Four: Usability Testing

13. Usability Testing (භාවිතයේ පහසුව ටෙස්ට් කිරීම)

- **Concept Explanation (Sinhala): Usability testing** මඟින් පරීක්ෂා කරන්නේ ඇප් එක සාමාන්‍ය මිනිසුන්ට තේරුම් ගැනීමට සහ භාවිතා කිරීමට කොතරම් පහසුද යන්නයි.

  වැදගත්ම දේ: ඔයාගේ React කෝඩ් එක තාක්ෂණිකව 100% ක් නිවැරදි වෙන්න පුළුවන් (බටන් එක ක්ලික් කළාම වැඩ කරයි). නමුත් යූසර්ට ඒ බටන් එක කොහෙද තියෙන්නේ කියලා හොයාගන්න බැරි නම්, එතැන තියෙන්නේ Usability ප්‍රශ්නයක් මිසක් කෝඩ් එකේ බග් එකක් නෙවෙයි.

  Usability Testing වල ප්‍රධාන වර්ග:

  - **Explorative testing:** ඇප් එක ඩිසයින් කරන මුල් අවධියේදීම රෆ් ලේඅවුට් (rough layout) පෙන්වා යූසර්ස්ලාගෙන් අදහස් විමසීම.
  - **Comparative testing:** ඩිසයින් ක්‍රම දෙකක් (Layouts දෙකක්) පෙන්වා වඩාත් පැහැදිලි එක තෝරාගැනීම.
  - **Assessment testing:** යූසර් කෙනෙකුට ඇප් එකේ වැඩක් (Task එකක්) තනියම අවසන් කරන්න දී ඔහු අතරමං වන තැන් නිරීක්ෂණය කිරීම.
  - **Validation testing:** අවසාන නිමි ඇප් එකේ සියලුම වැඩ කොටස් සාර්ථකව කළ හැකිදැයි තහවුරු කරගැනීම.

  Usability ටෙස්ට් එකක් කරන ආකාරය: යූසර්ස්ලා 5 දෙනෙකුට කෝස් එකක් සොයාගෙන Enroll වන ලෙස පවසන්න. ඔවුන් අතරමං වන, නතර වන තැන් නිහඬව නිරීක්ෂණය කර සටහන් කරගන්න. ඔවුන්ට අතරමැදදී උදව් කිරීමට හෝ ඉන්ටර්ෆේස් එක පැහැදිලි කිරීමට ඉක්මන් නොවන්න. එවිට ඔවුන් මුහුණ දෙන සැබෑ අපහසුතාවයන් ඔයාට හඳුනාගත හැක.

  Usability Tools: Loop11, Maze, Userbrain, UserTesting, UXTweak.

Part Five: Compatibility Testing

14. Compatibility Testing (ගැළපුම් ටෙස්ටිං)

- **Concept Explanation (Sinhala): Compatibility testing** මඟින් පරීක්ෂා කරන්නේ ඔයාගේ ඇප් එක ඔයාගේ ලැප්ටොප් එකෙන් සහ ඔයා පාවිච්චි කරන බ්‍රවුසර් එකෙන් පිටත, වෙනත් ඕනෑම පරිසරයක (environments) නිවැරදිව වැඩ කරනවාද යන්නයි.

  අවධානය යොමු කළ යුතු ප්‍රධාන ක්ෂේත්‍ර:

  - **Browser:** Chrome, Firefox, Safari, Edge. (උදා: Chrome වල ලස්සනට පෙනෙන layout එකක් Safari වල කැඩෙන්න පුළුවන්).
  - **OS (ඔපරේටිං සිස්ටම්):** Windows, macOS, Android, iOS.
  - **Hardware:** ඉතා ඉහළ ලැප්ටොප් වල මෙන්ම අඩු මිල අඩු ස්පෙස් (low-end) ෆෝන් වලත් වැඩද?
  - **Network:** වේගවත් Wi-Fi මෙන්ම ඉතා දුර්වල, ස්ලෝ මොබයිල් නෙට්වර්ක් වලත් ඇප් එක ලෝඩ් වෙනවාද?
  - **Mobile behavior:** Touch කරන්න ලෙහෙසිද? කීබෝඩ් එක ඕපන් වුණාම බටන් හැංගෙනවාද? Screen size එකට responsive ද?
  - **Input method:** Mouse, keyboard සහ Touch input වලට ගැලපෙනවාද?

  ආධුනිකයෙකු සඳහා මූලික උපක්‍රමය: අවම වශයෙන් Chrome desktop, Firefox desktop, Safari mobile, Chrome Android, සහ Keyboard navigation (Tab කී එකෙන් යාම) පරීක්ෂා කරන්න.

Part Six: Performance Testing

15. Performance Testing (වේගය සහ ස්ථාවරත්වය ටෙස්ට් කිරීම)

- **Concept Explanation (Sinhala): Performance testing** මඟින් පරීක්ෂා කරන්නේ ඇප් එකේ වේගය (Speed), ප්‍රතිචාර දැක්වීමේ කාලය (Responsiveness), වර්ධනය වීමේ හැකියාව (Scalability) සහ ස්ථාවරත්වය (Stability) යි.

  Usability සහ Performance පටලවා නොගන්න:

  - **Usability ප්‍රශ්නය:** "යූසර්ට සර්ච් බොක්ස් එක පාවිච්චි කරන හැටි තේරෙනවද?"
  - **Performance ප්‍රශ්නය:** "යූසර්ස්ලා 10,000ක් එකපාර ඇප් එකට ආවම සර්වර් එක ක්‍රෑෂ් නොවී වේගයෙන් වැඩ කරයිද?"

  Performance ටෙස්ට් වර්ග පහක්:

ටෙස්ට් වර්ගය (Type)	සරල තේරුම	උදාහරණය
Load Testing	සාමාන්‍ය හෝ බලාපොරොත්තු වන උපරිම ට්‍රැෆික් එකට ඇප් එක ඔරොත්තු දෙනවාදැයි බැලීම.	යූසර්ස්ලා 1,000ක් එකවර ඇප් එක පාවිච්චි කරන විට එය හොඳින් ක්‍රියා කරයිද?
Stress Testing	සිස්ටම් එකේ සාමාන්‍ය සීමාවන් පන්නා අන්තයටම ට්‍රැෆික් දී ක්‍රෑෂ් වන මට්ටම බැලීම.	යූසර්ස්ලා 50,000ක් එකවර ආවොත් සර්වර් එකට මොකද වෙන්නේ?
Soak Testing	දීර්ඝ වෙලාවක් එක දිගට සිස්ටම් එක රන් කරමින් මෙමරි ලීක්ස් (memory leaks) තිබේදැයි බැලීම.	ඇප් එක පැය 24ක් එක දිගට රන් වෙද්දී සර්වර් එක ස්ලෝ වෙනවාද?
Spike Testing	එකපාරටම ට්‍රැෆික් එක දැවැන්ත ලෙස වැඩි වී නැවත අඩු වන විට සිස්ටම් එක හැසිරෙන ආකාරය.	කෝස් එකක් ලෝන්ච් කළ සැනින් තත්පරයක් තුළ 5,000ක් ආවොත් සයිට් එක ඩවුන් වෙනවාද?
Breakpoint / Capacity Testing	සිස්ටම් එක ක්‍රෑෂ් වීමට හෝ අධික ලෙස ස්ලෝ වීමට පටන් ගන්නා නිවැරදිම සීමාව (limit) සෙවීම.	යූසර්ස්ලා ප්‍රමාණය 500, 1000, 2000, 5000 ලෙස ක්‍රමයෙන් වැඩි කරමින් බිඳ වැටෙන සීමාව සෙවීම.

Part Seven: Security Testing

16. Security Testing (ආරක්ෂණ ටෙස්ටිං)

- **Concept Explanation (Sinhala): Security testing** මඟින් පරීක්ෂා කරන්නේ ඇප් එක සහ එහි ඇති පරිශීලක දත්ත හැකින් ප්‍රහාර වලින්, අනවසර පිවිසුම් වලින් සහ දත්ත කාන්දුවීම් වලින් ආරක්ෂා වී ඇතිද යන්නයි.

  ආරක්ෂණයේ මූලධර්ම 6 (Principles):

  1. **Confidentiality (රහස්‍යභාවය):** තමන්ට අයිති නැති දත්ත වෙනත් අයට පෙනෙන්නේ නැත.
  2. **Integrity (අඛණ්ඩතාව/නොවෙනස් බව):** අනවසරයෙන් දත්ත වෙනස් කළ නොහැක (උදා: වෙනත් යූසර් කෙනෙකුගේ කෝස් ප්‍රගතිය වෙනස් කළ නොහැක).
  3. **Authentication (හඳුනා ගැනීම):** යූසර් කවුදැයි තහවුරු කරගැනීම (උදා: Login වීම).
  4. **Authorization (අවසරය):** ලොග් වූ යූසර්ට කරන්න අවසර ඇත්තේ මොනවාදැයි බැලීම (උදා: Admin කෙනෙකුට පමණක් `/admin` පිටුවට යාමට දීම).
  5. **Availability (භාවිතයට තිබීම):** හැකර් ප්‍රහාර මැද වුවද සැබෑ පරිශීලකයින්ට ඇප් එක දිගටම පාවිච්චි කිරීමට හැකිවීම.
  6. **Non-repudiation (ප්‍රතික්ෂේප නොකිරීම):** යූසර් කෙනෙකු කළ ක්‍රියාවක් පසුව ඔහු කළේ නැතැයි බොරු කිව නොහැකි වන ලෙස වැදගත් සිදුවීම් සටහන් කර තැබීම (Logging).

  ප්‍රධාන ආරක්ෂක තර්ජන (Threats):

  - **XSS (Cross-Site Scripting):** ඇටැකර් කෙනෙකු විසින් වෙනත් යූසර් කෙනෙකුගේ බ්‍රවුසර් එකක හානිකර ජාවාස්ක්‍රිප්ට් කෝඩ් රන් කරවීම (උදා: කමෙන්ට් box එකක් හරහා කෝඩ් ඇතුල් කිරීම).
  - **SQL Injection:** අනවසර ඉන්පුට් දමා ඩාටාබේස් එකේ කොයරි (Queries) වෙනස් කර දත්ත සොරකම් කිරීම.
  - **DoS (Denial of Service):** සර්වර් එකට දරාගත නොහැකි තරම් ව්‍යාජ රික්වෙස්ට් එවා සැබෑ යූසර්ස්ලාට ඇප් එක අකර්මණ්‍ය කිරීම.

  වැදගත්ම රීතිය: Frontend ආරක්ෂාව පමණක් ප්‍රමාණවත් නොවේ! React වලින් "Admin" බටන් එකක් හෝ ලින්ක් එකක් හංගලා තිබුණු පලියට ඇප් එක සෙකියුර් වෙන්නේ නැහැ. ඕනෑම කෙනෙකුට කෝඩ් එක bypass කර API රික්වෙස්ට් එකක් යැවිය හැක. එබැවින් සැබෑ සෙකියුරිටි චෙක් එක සිදුවිය යුත්තේ **Backend** එකෙනි.

  Security Tools වර්ග:

  - **SAST (Static Application Security Testing):** කෝඩ් එක රන් කරන්නේ නැතිව, කෝඩ් එක කියවා බලා එහි ඇති දුර්වලතා සෙවීම (Static scanning).
  - **DAST (Dynamic Application Security Testing):** රන් වන ඇප් එකකට පිටතින් ව්‍යාජ ප්‍රහාර එල්ල කරමින් දුර්වලතා සෙවීම (Dynamic scanning).
  - **Penetration Testing (පෙන් ටෙස්ටිං):** සැබෑ සෙකියුරිටි විශේෂඥයෙකු විසින් හැකර් කෙනෙකු සේ සිතා ඇප් එකට ඇතුල් වීමට උත්සාහ කර දුර්වලතා සෙවීම.

Part Eight: A/B, Alpha, and Beta Testing

17. A/B Testing (ඒ/බී ටෙස්ටිං)

- **Concept Explanation (Sinhala): A/B testing** මඟින් කරන්නේ ඩිසයින් හෝ විශේෂාංගයක සංස්කරණ දෙකක් (Version A සහ Version B) සාදා, ඒවා යූසර්ස්ලාට පෙන්වා, ලැබෙන දත්ත (data/metrics) අනුව වඩාත්ම සාර්ථක එක තෝරාගැනීමයි.

  උදාහරණය:

  - **Version A (බටන් එකේ ලේබලය):** "Start Learning"
  - **Version B (බටන් එකේ ලේබලය):** "View Courses"
  - **Success Metric:** Click-through rate (වැඩිපුරම ක්ලික් වන බටන් එක).
  - **Decision:** වැඩිපුරම ක්ලික් ලැබෙන බටන් එක ස්ථිර ඩිසයින් එක ලෙස තෝරාගැනීම.

  මතක තබා ගන්න: A/B testing යනු කෝඩ් එකේ බග් හෝ සෙකියුරිටි ප්‍රශ්න බලන එකක් නොවේ. එය නිෂ්පාදනයේ (product) සාර්ථකත්වය මනින ක්‍රමයකි.

  Tools: GrowthBook, LaunchDarkly.

18. Alpha Testing (ඇල්ෆා ටෙස්ටිං)

- **Concept Explanation (Sinhala): Alpha testing** යනු ඇප් එකක් සාමාන්‍ය ජනතාවට මුදාහැරීමට පෙර, තමන්ගේම ආයතනයේ සිටින ඩිවෙලොපර්ස්ලා, QA ටෙස්ටර්ස්ලා හෝ තෝරාගත් කුඩා පිරිසක් ලවා ඉතා මුල් අවධියේදී සිදුකරන අභ්‍යන්තර ටෙස්ටිං ක්‍රියාවලියකි. මෙයින් ප්‍රධානතම බග්ස් (major bugs) මුලින්ම අල්ලා ගනී.

19. Beta Testing (බීටා ටෙස්ටිං)

- **Concept Explanation (Sinhala): Beta testing** යනු ඇප් එක සම්පූර්ණයෙන්ම ලෝන්ච් කිරීමට පෙර, සැබෑ පරිශීලකයින්ගෙන් (real users) තෝරාගත් සීමිත පිරිසකට (උදා: යූසර්ස්ලා 50කට) පමණක් ඇප් එක පාවිච්චි කිරීමට දී ඔවුන්ගෙන් සැබෑ ලෝකයේ ප්‍රතිපෝෂණ (real-world feedback) ලබාගන්නා අවසාන ටෙස්ටිං පියවරයි.

Part Nine: Regression Testing and Retesting

20. Regression Testing (රිග්‍රෙෂන් ටෙස්ටිං)

- **Concept Explanation (Sinhala): Regression testing** මඟින් කරන්නේ ඇප් එකට අලුත් කෝඩ් එකක් දැමූ විට හෝ වෙනස්කමක් කළ විට, කලින් හොඳට වැඩ කරපු පරණ විශේෂාංග (existing features) කැඩී ඇත්දැයි ආපහු පරීක්ෂා කිරීමයි.

  උදා: ඔයා කෝස් කාඩ් එකේ ඩිසයින් එක වෙනස් කරනවා. ඊට පස්සේ ඔයා ආපහු කෝස් සර්ච් එක, ලොගින් එක, සහ එන්රෝල් වෙන එක ටෙස්ට් කරලා බලන්න ඕනේ. මොකද අලුත් වෙනස්කම නිසා නොදැනුවත්වම පරණ කෝඩ් එකක් කැඩෙන්න පුළුවන්.

21. Retesting (රී-ටෙස්ටිං)

- **Concept Explanation (Sinhala): Retesting** යනු ඇප් එකේ තිබී හමු වූ නිශ්චිත බග් (bug) එකක් ඩිවෙලොපර් විසින් ෆික්ස් (fix) කළ පසු, එම බග් එක ඇත්තටම හරියාකාරව විසඳී ඇත්දැයි බැලීමට එම අසාර්ථක වූ ටෙස්ට් එකම නැවත සිදු කිරීමයි.

  ප්‍රධාන වෙනස වටහා ගන්න:

  - **Retesting:** "මේ සොයාගත් බග් එක දැන් හරියට හැදිලද?" කියා බැලීම (Focuses on the fix).
  - **Regression Testing:** "මේ කරපු අලුත් වෙනස්කම නිසා වෙනත් පරණ වැඩ කරන ෆීචර්ස් කැඩිලද?" කියා බැලීම (Focuses on unexpected breakage).

Part Ten: Course Browser Test Strategy (උදාහරණ ඇප් එකක ටෙස්ටිං උපාය මාර්ගය)

- **Concept Explanation (Sinhala):** අපි හදන Course Browser ඇප් එකක් සඳහා නිවැරදිව ටෙස්ට් වර්ග ගළපන ආකාරය පහත වගුවෙන් දැක්වේ:

සැබෑ අවස්ථාව (Scenario)	ගැළපෙනම ටෙස්ට් වර්ගය (Best Test Type)
`formatPrice(4)` දුන් විට එය `$4.00` ලෙස සකසනවාද?	**Unit Test** (කුඩා pure function එකක් නිසා)
Search box එකේ "React" කියා සර්ච් කළ විට React කෝස් පමණක් පෙන්වනවාද?	**Functional Test** (ෆීචර් එකක හැසිරීමක් නිසා)
ඇප් එක ඩිප්ලෝයි කළ සැනින් Home, Login සහ Course List පිටු ක්‍රෑෂ් නොවී ලෝඩ් වෙනවාද?	**Smoke Test** (ඇප් එක පණ තියෙනවාද කියා බලන මූලික චෙක් එකක් නිසා)
යූසර් ලොග් වී, කෝස් එකක් තෝරා, Enroll වී, සාර්ථක පණිවිඩය දකින තුරු සම්පූර්ණ ක්‍රියාවලිය සිදුවේද?	**E2E Test** (සම්පූර්ණ පරිශීලක ගමනක් නිසා)
කෝස් කාඩ් එක iPhone Safari සහ Chrome Android යන දෙකෙහිම ලස්සනට පෙනෙනවාද?	**Compatibility Test** (විවිධ බ්‍රවුසර්/උපාංග නිසා)
කෝස් 10,000ක් තිබුණත් සර්ච් එක ස්ලෝ නොවී එක වර වැඩ කරනවාද?	**Performance Test** (වේගය සහ ධාරිතාව නිසා)
ලොග් අවුට් වූ කෙනෙකුට කෙලින්ම `/dashboard` URL එක ගසා ඇතුල් විය හැකිද?	**Security Test** (අවසර සහ දත්ත ආරක්ෂාව නිසා)
ඩිසයින් එක වෙනස් කළ පසු පරණ Enroll බටන් එක තවමත් වැඩ කරනවාද?	**Regression Test** (අලුත් වෙනස්කමක් නිසා පරණ දේ කැඩුනාදැයි බැලීමට)
ලොගින් එකේ තිබූ බග් එක හැදූ පසු දැන් ලොගින් එක වැඩ කරනවාද?	**Retesting** (නිශ්චිත බග් එක හැදුනාදැයි බැලීමට)

Part Eleven: QA Plan Component (Mini-Project)

- **Concept Explanation (Sinhala):** මෙම Mini-project එකෙන් කරන්නේ අපි ඉගෙනගත් සියලුම ටෙස්ට් වර්ග, ඒවායේ අවදානම් (risks) සහ උදාහරණ (examples) ලස්සන React List එකක් ලෙස `map()` භාවිතයෙන් සහ ස්ථාවර `key` එකක් සහිතව ස්ක්‍රීන් එකේ පෙන්වීමයි.
- **Code Block:**

```
import React from 'react';

const qaCategories = [
  {
    type: "Unit Test",
    risk: "Small logic may calculate the wrong result",
    example: 'formatPrice(4) should return "$4.00"',
  },
  {
    type: "Functional Test",
    risk: "A feature may not behave correctly",
    example: "Search should show matching courses",
  },
  {
    type: "Smoke Test",
    risk: "The app may be obviously broken after deployment",
    example: "Home, Login, and Course List pages open without crashing",
  },
  {
    type: "E2E Test",
    risk: "A critical user journey may break",
    example: "User logs in, opens a course, enrolls, and sees success",
  },
  {
    type: "Usability Test",
    risk: "Users may not understand how to use the app",
    example: "Watch users try to find and enroll in a React course",
  },
  {
    type: "Compatibility Test",
    risk: "The app may break on some browsers or devices",
    example: "Course cards work on iPhone Safari and Chrome Android",
  },
  {
    type: "Performance Test",
    risk: "The app may become slow under traffic or large data",
    example: "Search stays responsive with 10,000 courses",
  },
  {
    type: "Security Test",
    risk: "Private data or protected routes may be exposed",
    example: "Logged-out users cannot open /dashboard directly",
  },
  {
    type: "A/B Test",
    risk: "We may not know which design performs better",
    example: 'Compare "Start Learning" vs "View Courses"',
  },
  {
    type: "Alpha Test",
    risk: "Major issues may exist before public release",
    example: "Selected testers try the app before release",
  },
  {
    type: "Beta Test",
    risk: "Real users may find issues before full launch",
    example: "A small group of real learners uses the app before public
launch",
  },
  {
    type: "Regression Test",
    risk: "New changes may break old behavior",
    example: "After changing course cards, rerun search and enroll
tests",
  },
  {
    type: "Retesting",
    risk: "A known bug fix may not actually work",
    example: "After fixing login, test the same failed login case again",
  }
];

export default function QAPlan() {
  return (
    <main style={{ padding: '20px', fontFamily: 'sans-serif', maxWidth:
'800px', margin: '0 auto' }}>
      <h1 style={{ color: '#2c3e50' }}>QA Plan for Course Browser</h1>
      <p style={{ color: '#7f8c8d', fontSize: '1.1rem' }}>
        This plan matches each test type to the risk it is designed to
catch.
      </p>

      <section style={{ display: 'grid', gap: '20px', marginTop: '30px'
}}>
        {qaCategories.map((item) => (
          <article
            key={item.type}
            style={{
              padding: '15px',
              border: '1px solid #e2e8f0',
              borderRadius: '8px',
              backgroundColor: '#f8fafc',
              boxShadow: '0 2px 4px rgba(0,0,0,0.02)'
            }}
          >
            <h2 style={{ margin: '0 0 10px 0', color: '#3182ce' }}>
{item.type}</h2>
            <p style={{ margin: '5px 0' }}>
              <strong>Risk:</strong> {item.risk}
            </p>
            <p style={{ margin: '5px 0', color: '#4a5568' }}>
              <strong>Example:</strong> <code style={{ backgroundColor:
'#edf2f7', padding: '2px 6px', borderRadius: '4px' }}>{item.example}
</code>
            </p>
          </</article>
        ))}
      </section>
    </main>
  );
}
```

Part Twelve & Thirteen: Common Beginner Mistakes & Revision (ආධුනිකයින් කරන ප්‍රධාන වැරදි)

- **Concept Explanation (Sinhala):** මෙම වැරදි වටහා ගැනීම ඔයාගේ විභාග සඳහා ඉතා වැදගත් වේ:
  - **වැරැද්ද 1: හැම දේටම E2E පාවිච්චි කිරීම.**
    - වැරදි: `formatPrice()` වැනි සරල function එකක් ටෙස්ට් කරන්න E2E ලියා කාලය නාස්ති කිරීම.
    - නිවැරදි: සරල pure functions සඳහා **Unit tests** ද, ප්‍රධාන යූසර් ජර්නි සඳහා පමණක් **E2E** ද පාවිච්චි කරන්න.
  - **වැරැද්ද 2: Usability සහ Performance පටලවා ගැනීම.**
    - වැරදි: සර්වර් ලෝඩ් එක මනින්න Usability testing පාවිච්චි කිරීම.
    - නිවැරදි: සර්වර් ලෝඩ් එකට **Performance/Load testing** ද, පාවිච්චි කිරීමේ පහසුවට **Usability testing** ද පාවිච්චි කරන්න.
  - **වැරැද්ද 3: සෙකියුරිටි ප්‍රශ්න සෙවීමට A/B Testing කිරීම.**
    - වැරදි: SQL Injection තිබේදැයි බලන්න A/B Testing කිරීම.
    - නිවැරදි: SQL Injection සෙවීමට **Security Testing** ද, බටන් වල සාර්ථකත්වය බැලීමට **A/B Testing** ද පාවිච්චි කරන්න.
  - **වැරැද්ද 4: Smoke Test එකෙන් මුළු ඇප් එකම පරිපූර්ණ යැයි සිතීම.**
    - වැරදි: Smoke test එක පාස් වූ නිසා තවදුරටත් ගැඹුරින් ටෙස්ට් නොකිරීම.
    - නිවැරදි: Smoke test එකෙන් බලන්නේ ඇප් එක ක්‍රෑෂ් නොවී පණ තිබේද යන්න පමණි. ගැඹුරු තර්කන (logic) සඳහා වෙනත් ටෙස්ට් කළ යුතුය.
  - **වැරැද්ද 5: Frontend සෙකියුරිටි පමණක් විශ්වාස කිරීම.**
    - වැරදි: React වලින් "Admin" බටන් එක හැංගූ නිසා සෙකියුරිටි එක 100% ක් සාර්ථක යැයි සිතීම.
    - නිවැරදි: යහපත් පරිශීලක අත්දැකීමක් (UX) සඳහා බටන් හැංගුවද, සැබෑ සෙකියුරිටි එක Backend එකෙන්ම තහවුරු කළ යුතුය.

Part Seventeen: One-Page Memory Summary (එක් පිටුවක මතක සාරාංශය)

- **Concept Explanation (Sinhala):** ටෙස්ටිං උපාය මාර්ගයේ රන්වන් රීතිය (Golden Rule):

  > **අහඹු ලෙස ටෙස්ට් කරන්න එපා! >**
  >
  > 1. මුලින්ම සිතන්න: "මෙහිදී වැරදිය හැක්කේ කුමක්ද?" (What can go wrong?)
  >
  > 2. එය කුමන අවදානමක්දැයි හඳුනාගන්න (Identify the Risk).
  >
  > 3. එම අවදානම හොඳින්ම මඟහරවා ගත හැකි ටෙස්ට් වර්ගය තෝරන්න (Select Best Test Type).